



Layout managers: ubicar componentes sin coordenadas

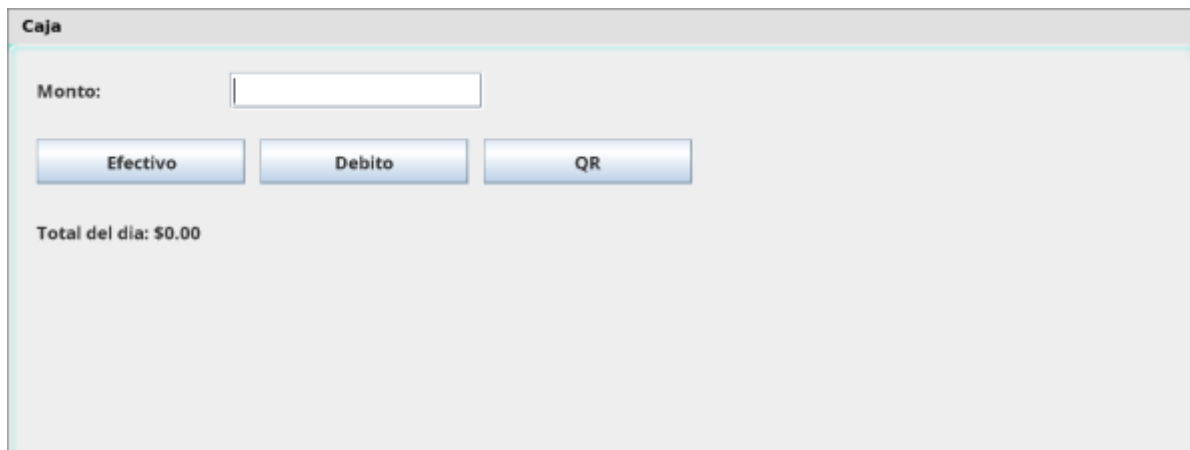
El problema

Cada componente de la caja tiene sus coordenadas escritas a mano con `setBounds()`, y para que entre un tercer botón (QR) hay que ensanchar la ventana y correr los otros dos (Efectivo y Debito):

```
this.ventana.setSize(480, 200);           // antes: 360
// ...
this.botonEfectivo.setBounds(20, 65, 140, 30); // antes: (40, 65, 130, 30)
this.botonDebito.setBounds(170, 65, 140, 30);  // antes: (190, 65, 130, 30)
this.botonQR.setBounds(320, 65, 140, 30);
```

Un cambio visual implica modificar las coordenadas de los componentes visuales.

El segundo problema aparece al sacar `setResizable(false)` y agrandar la ventana:



La caja con el QR, después de agrandar la ventana. Cada componente sigue en el píxel que dice su `setBounds()`.

Al modificar el tamaño de la ventana los componentes visuales no se redimensionan ni se reubican, porque fueron puestos en coordenadas específicas. Las dos cosas se resuelven con el mismo mecanismo: la vista define la disposición relativa de los componentes (qué va arriba, qué va al lado de qué) y un objeto calcula las coordenadas en píxeles cada vez que la ventana cambia.

Layout manager

Todo contenedor de Swing (`JFrame`, `JPanel`) tiene un layout manager: el objeto que calcula la posición y el tamaño de los componentes agregados. `setLayout(null)` lo elimina, y por eso la vista tuvo que escribir coordenadas.

Un layout manager implementa la interfaz `LayoutManager`. El contenedor invoca sus métodos al mostrarse, al cambiar de tamaño y al agregar un componente:



```
public interface LayoutManager {  
    void layoutContainer(Container contenedor);           // ubica y dimensiona cada hijo  
    Dimension preferredLayoutSize(Container contenedor); // tamaño que necesita el  
    contenedor  
    // ... y tres mas  
}
```

El contenedor depende de la interfaz, no de una implementación, igual que Caja con Notificador. Java trae varias (FlowLayout, BorderLayout, GridLayout) y setLayout() elige cuál.

El cálculo usa el tamaño preferido de cada componente (getPreferredSize()). En un JButton se calcula a partir del texto y la fuente actual. En un JTextField se calcula a partir de las columnas indicadas al crearlo (new JTextField(12)); sin columnas, el ancho preferido es casi cero.

El layout manager calcula las coordenadas cada vez que el contenedor cambia. La vista no las escribe.

FlowLayout

Ubica los componentes en una fila, en el orden del add(), cada uno con su tamaño preferido; cuando la fila se llena, continúa en la siguiente. Es el layout manager por defecto de JPanel. Para la fila de arriba de la caja (etiqueta y campo):

```
JPanel panelMonto = new JPanel(new FlowLayout(FlowLayout.LEFT));  
panelMonto.add(etiquetaMonto);  
panelMonto.add(this.campoMonto); // creado con new JTextField(12)
```

FlowLayout.LEFT alinea la fila a la izquierda (también hay CENTER y RIGHT). Sirve para filas cortas, no para una ventana entera.

FlowLayout: en fila, en el orden del add(), cada uno con su tamaño preferido.

BorderLayout

Es el layout manager por defecto de JFrame. Divide el contenedor en cinco regiones (norte, sur, este, oeste, centro) y el segundo parámetro de add() indica la región. Norte y sur ocupan todo el ancho, con su altura preferida; este y oeste, su ancho preferido y la altura restante; el centro, todo el espacio restante.

Primer intento: sacar setLayout(null) y repartir los componentes en las regiones:

```
this.ventana.add(etiquetaMonto, BorderLayout.NORTH);  
this.ventana.add(this.campoMonto, BorderLayout.CENTER);  
this.ventana.add(this.botonEfectivo, BorderLayout.WEST);  
this.ventana.add(this.botonDebito, BorderLayout.EAST);  
this.ventana.add(this.etiquetaTotal, BorderLayout.SOUTH);
```



BorderLayout sin paneles. El campo ocupa todo el centro y los botones son tan altos como la ventana. Para el QR no quedó región.

El campo (centro) ocupa todo el espacio restante, sin importar su tamaño preferido. Los botones (este y oeste) toman toda la altura restante. Y el QR no está: cada región admite un solo componente, y un segundo `add()` en la misma región reemplaza al primero.

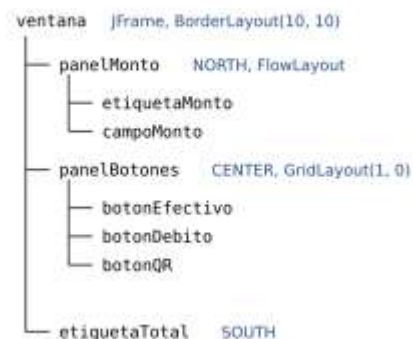
BorderLayout: cinco regiones, un componente por región. El centro ocupa el espacio restante.

Paneles anidados

Un `JPanel` es un componente (se agrega a una región) y a la vez un contenedor (tiene su propio layout manager). En cada región de la ventana va un panel, y cada panel dispone sus componentes con su propio layout:



Quién contiene a quién



La ventana y sus paneles. Cada contenedor tiene su layout manager, que ubica solo a sus hijos directos.

El constructor de la vista:



```
public VistaCaja() {
    this.ventana = new JFrame("Caja");
    this.ventana.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
    this.ventana.setLayout(new BorderLayout(10, 10)); // 10 px entre regiones

    JLabel etiquetaMonto = new JLabel("Monto:");
    this.campoMonto = new JTextField(12);
    this.botonEfectivo = new JButton("Efectivo");
    this.botonDebito = new JButton("Debito");
    this.botonQR = new JButton("QR");
    this.etiquetaTotal = new JLabel("Total del día: $0.00");

    JPanel panelMonto = new JPanel(new FlowLayout(FlowLayout.LEFT));
    panelMonto.add(etiquetaMonto);
    panelMonto.add(this.campoMonto);

    JPanel panelBotones = new JPanel(new GridLayout(1, 0, 8, 0));
    panelBotones.setBorder(BorderFactory.createEmptyBorder(0, 10, 0, 10));
    panelBotones.add(this.botonEfectivo);
    panelBotones.add(this.botonDebito);
    panelBotones.add(this.botonQR);

    this.etiquetaTotal.setBorder(BorderFactory.createEmptyBorder(0, 10, 10, 10));

    this.ventana.add(panelMonto, BorderLayout.NORTH);
    this.ventana.add(panelBotones, BorderLayout.CENTER);
    this.ventana.add(this.etiquetaTotal, BorderLayout.SOUTH);

    this.ventana.pack();
    this.ventana.setLocationRelativeTo(null);
}
```

Sin `setBounds()` ni `setSize()`. `pack()` dimensiona la ventana según el tamaño preferido de sus paneles, y cada panel según el de sus componentes. `setLocationRelativeTo(null)` va después de `pack()`, porque centra la ventana usando su tamaño. `createEmptyBorder()` es un margen: un borde que no dibuja nada.

Agregar el QR requiere crear el botón y un `add()`: `GridLayout` reparte el panel en tres celdas iguales en vez de dos.

Una región, un panel. Adentro del panel, su propio layout.

El tamaño también cuenta

Al agrandar la ventana, los layouts recalculan: el centro ocupa el espacio restante y `GridLayout` lo reparte entre los botones. El texto, no. Un layout manager calcula posición y tamaño de los componentes; la fuente queda en los 12 puntos con que se creó cada uno:



La vista con los paneles, después de agrandar la ventana. Los botones crecen, el texto no.

Un resultado visual agradable depende de la ubicación y también del tamaño de los textos y los campos, y con layouts solos a veces no se logra. Para esta caja, pensada para una pantalla táctil, la fuente se escala con la ventana. Un `ComponentAdapter` recibe el evento de cambio de tamaño, y el método calcula el factor entre el tamaño actual y el de `pack()`:

```
this.ventana.addComponentListener(new ComponentAdapter() {
    @Override
    public void componentResized(ComponentEvent evento) {
        escalarFuentes();
    }
});

private void escalarFuentes() {
    float factor = Math.min((float) this.ventana.getHeight() / this.altoInicial,
        (float) this.ventana.getWidth() / this.anchoInicial);
    float tamano = this.fuenteInicial * Math.max(1f, factor);
    escalar(this.ventana.getContentPane(), tamano); // setFont() en cada componente
    this.ventana.revalidate(); // Los layouts recalculan
}
```

`escalar()` recorre los componentes y llama a `setFont()` en cada uno con el tamaño nuevo. Como el tamaño preferido depende de la fuente, `revalidate()` hace que los layouts recalculen con los tamaños nuevos:



La misma vista, al tamaño que eligió pack() y después de agrandar la ventana, con las fuentes escaladas.

El layout manager ubica y dimensiona los componentes. El tamaño del texto se decide aparte.

GridLayout y otros layouts

`GridLayout(filas, columnas, hgap, vgap)` divide el contenedor en celdas iguales y ubica un componente por celda, en el orden del `add()`; un 0 significa “las que hagan falta”. `new GridLayout(1, 0, 8, 0)` es una fila con 8 píxeles entre columnas. Las celdas ocupan todo el panel, por eso los botones crecen con la ventana. Con `FlowLayout` en el panel de botones, cada uno queda con su tamaño preferido, en una fila centrada arriba, y no crece. Cambiar uno por otro es cambiar el objeto que recibe `new JPanel(...)`; los `add()` no cambian.

Hay más: `BoxLayout` (una sola fila o columna), `GridBagLayout` (grilla con celdas de distinto tamaño, el más flexible y el más largo de escribir) y `CardLayout` (un componente visible por vez). Con `BorderLayout`, `FlowLayout`, `GridLayout` y paneles adentro de paneles se construye casi cualquier ventana de escritorio.

Cambiar la disposición de un panel es cambiar un objeto. El resto de la vista no cambia.



Qué se ganó

Cambió una clase, VistaCaja. ControladorCaja se registra en `getBotonEfectivo()`, `getBotonDebito()` y `getBotonQR()`, y esos métodos no cambiaron. Caja no tiene ninguna referencia a la vista. Un cambio en la disposición de la pantalla queda adentro de la vista.

Bibliografía sugerida para profundizar

Deitel, P., & Deitel, H. (2017). *Java How to Program, Early Objects*.

- Capítulo 26, “Swing GUI Components: Part 1”, las secciones sobre layout managers (FlowLayout, BorderLayout, GridLayout) y sobre el uso de paneles para armar diseños más complejos.

Aguilar, L. J. (2011). *Programación en Java: algoritmos, programación orientada a objetos e interfaz gráfica de usuario*. McGraw-Hill.

- Capítulo 20, “Gráficos I”, para los gestores de diseño y los contenedores.

Belmonte Fernández, O. (2012). *Desarrollo de proyectos informáticos con tecnología Java*. Universitat Jaume I.

- Capítulo 11, “Interfaces gráficas de usuario”, la sección sobre gestores de disposición. Gratuito y breve.

Oracle. *The Java Tutorials*, “Laying Out Components Within a Container”.

- Referencia oficial, gratuita. “A Visual Guide to Layout Managers” tiene una captura de cada layout y una página por cada uno con su código.